Procedurálne programovanie

7. Prednáška

Viacrozmerné polia

Anna Bou Ezzeddine

Obsah

1. opakovanie
2. viacrozmerné polia
3. príklady

Dané je pole **a** obsahujúce **n** celých čísel (typ **int**) usporiadaných od najmenšieho po najväčšie, celé číslo **cislo**, ktoré do poľa vkladáme, a celé číslo **kapacita** (typu **int**) zodpovedajúce dĺžke resp. veľkosti poľa (**n <= kapacita**).

Napíšte funkciu **vloz()**, ktorá vloží prvok **cislo** do poľa **a[]**, výsledné pole musí zostať usporiadané. Predpokladajte, že pole **a[]** nemusí byť dostatočné veľké pre ďalšie číslo. V prípade úpravy poľa **a** musí funkcia prostredníctvom výstupných argumentov upraviť hodnoty **n** a **kapacita**.

```
void vloz(int cislo, int **a, int *n, int *kapacita)
{
  int i;
  if ((*n) + 1 > *kapacita)
  {
    *kapacita += 10;
    *a = realloc(*a, *kapacita * sizeof(int));
  }
  for (i = (*n) - 1; i >= 0; i--)
  {
    if ((*a)[i] < cislo)
      break;
    (*a)[i + 1] = (*a)[i];
  }
  (*a)[i + 1] = cislo;
  (*n)++;
}
```

Generovanie pseudonáhodných čísel

- Generovanie pseudonáhodných čísel realizujeme v C štandardne pomocou funkcií **`rand()`** a **`srand()`**. Je potrebné vložiť súbory `stdlib.h` a `time.h`
- Inicializácia generátora náhodných čísel:
  - **srand((unsigned) time(NULL));**
- Ak túto funkciu nepoužijeme generovaná postupnosť je pri každom spustení programu rovnaká.
- Generovanie pseudonáhodných čísel:
  - **rand();**

Generovanie pseudonáhodných čísel

//Do **x1** je uložené pseudonáhodné číslo**(int)** z rozsahu **0** až **RAND_MAX**

```
x1 = rand();
```

//Do **x2** je uložené pseudonáhodné číslo**(int)** z rozsahu **0** až **99**

```
x2 = rand()%100;
```

//Do **x3** je uložené pseudonáhodné číslo**(double)** z intervalu **<0;1>**

```
x3 = (double) rand()/RAND_MAX;
```

//Do **x4** je uložené pseudonáhodné číslo**(double)** z intervalu **<a;b>**

```
x4 = a + (b-a)*(double) rand()/RAND_MAX;
```

P7.1

```c
#include <stdio.h>
// knižnica potrebna ku generatoru
#include <time.h>

int main(void) {
  int i, nahodne_cislo;

// inicializacia generatora nahodnych cisiel
  srand( time( 0 ) );
  printf ("Generovane cisla su: \n");

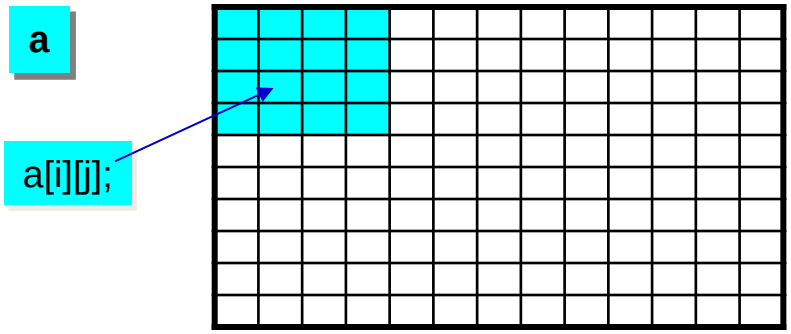
  for (i=0; i<=20; i++) {
  // /* generovanie cisla v rozsahu 1 - 100 */
    nahodne_cislo = rand()%100 + 1;
    printf ("%d \n",nahodne_cislo);
  }
  return 0;
}
```

Rôzne spôsoby definície dvojrozmerných polí

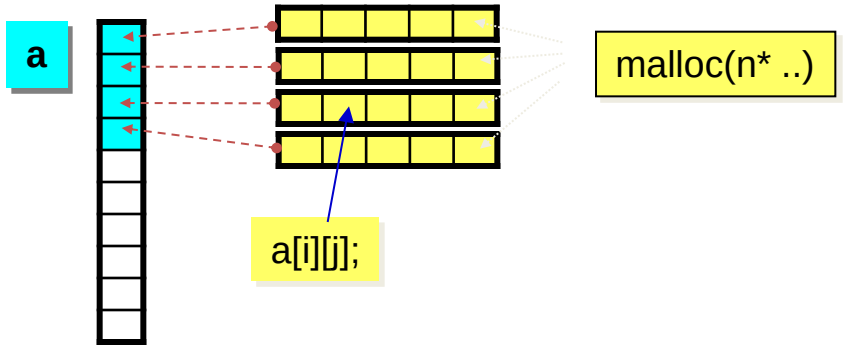
1. statické dvojrozmerné pole
2. pole smerníkov
3. smerník na pole
4. smerník na smerník

Rôzne definície dvojrozmerných polí – (statická, dynamická alokácia)

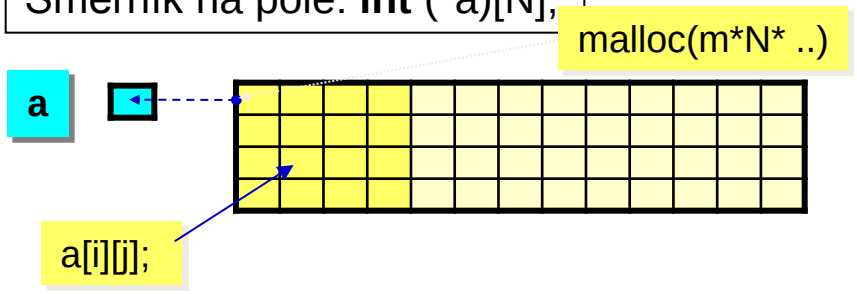
Statické pole: **int** a[M][N];



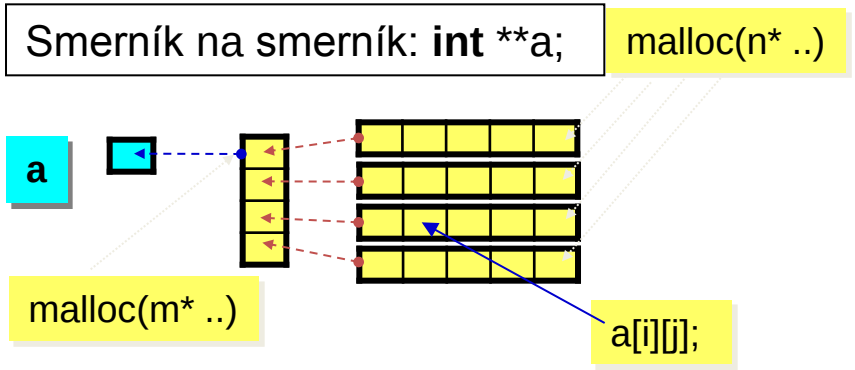
Pole smerníkov: **int** *a[M];



Smerník na pole: **int** (*a)[N];



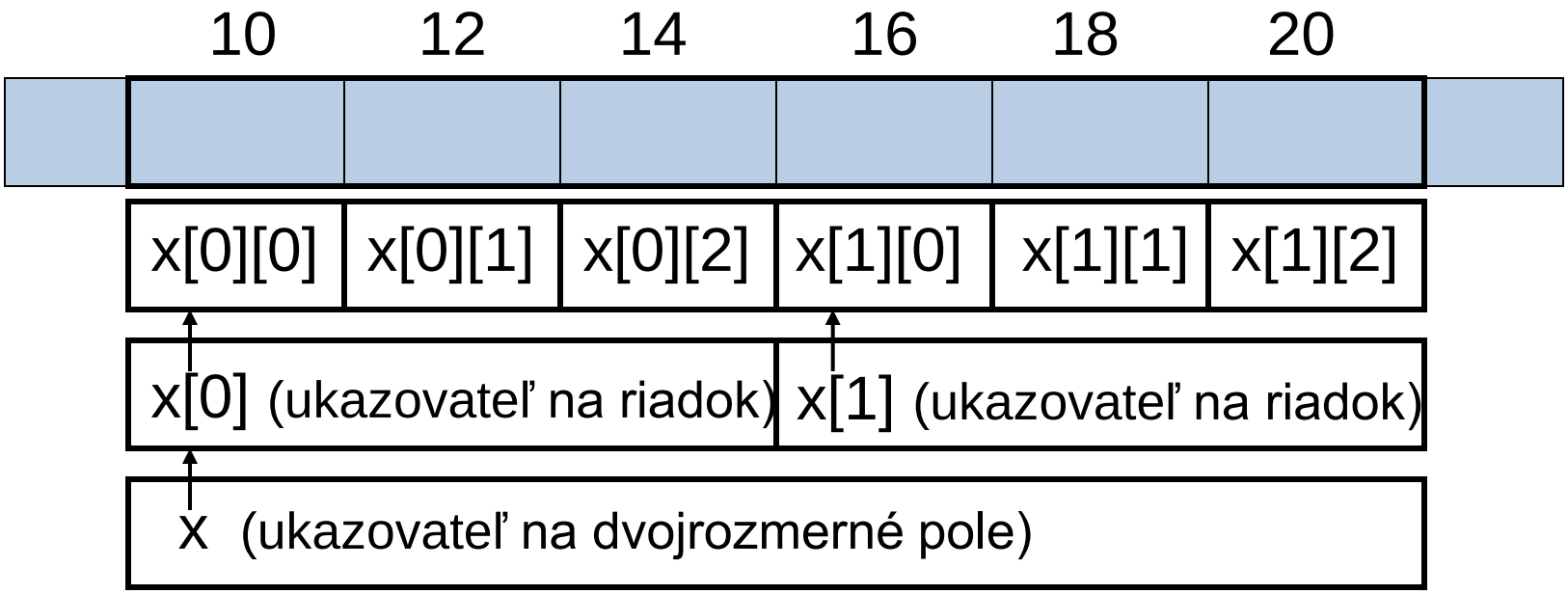
Smerník na smerník: **int** **a;



Uloženie 2D poľa v pamäti

- statické pole - po riadkoch

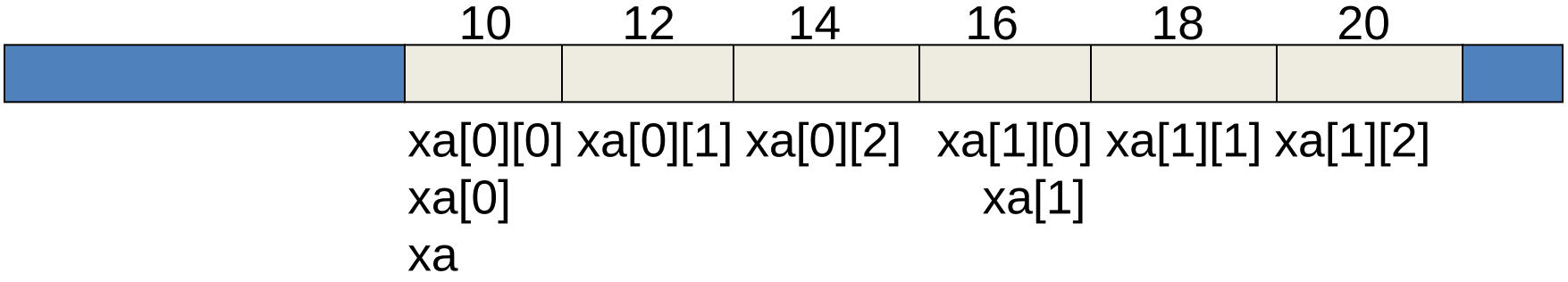
```
int x[2][3];
```



Statické dvojrozmerné pole

```
int xa[2][3];
```

- pole **`xa`**:
  - alokované pri preklade
  - súvislý blok 6 prvkov
  - uložené po riadkoch
  - konštantný smerník



Statické dvojrozmerné pole P7.2

```
#include <stdio.h>

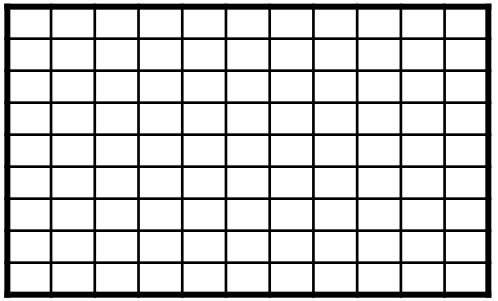
void vypis(int a[][3], int n, int m)
{
  int i, j;
  for (i = 0; i < n; i++)
  for (j = 0; j < m; j++)
    printf("%d ", a[i][j]);
}

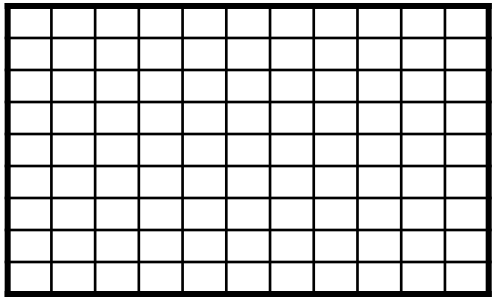
int main(void){
  int p[][3] = { {1,2,3}, {4,5,6}, {7,8,9}, {10,11,12} };

  vypis(p, 4, 3); // 1 2 3 4 5 6 7 8 9 10 11 12
  return 0;
}
```

Dvojrozmerné pole , textový súbor – zápis a načítanie - príklad

```
#define M 100
#define N 100
int a[M][N],b[M][N];
```

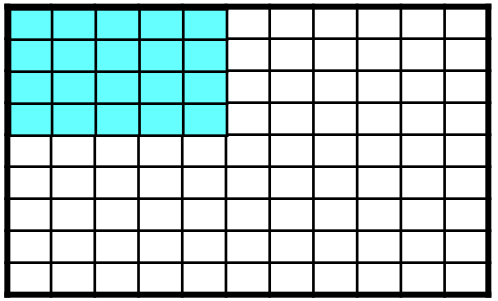


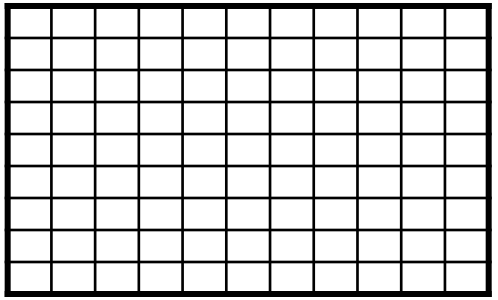


Dvojrozmerné pole , textový súbor – zápis a načítanie - príklad

```
#define M 100
#define N 100
int a[M][N],b[M][N];
```

```
FILE *fw;
int m,n,i,j;
scanf("%d %d",&m,&n);
for(i=0;i<m;i++)
 for(j=0;j<n;j++)
   a[i][j]=rand()%200-100;
```

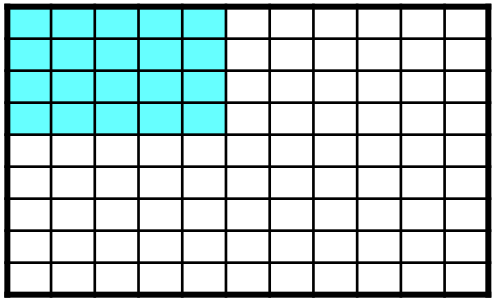




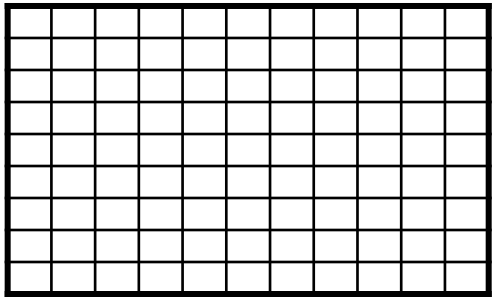
Dvojrozmerné pole , textový súbor – zápis a načítanie - príklad

```
#define M 100
#define N 100
int a[M][N],b[M][N];
```

```
FILE *fw;
int m,n,i,j;
scanf("%d %d",&m,&n);
for(i=0;i<m;i++)
 for(j=0;j<n;j++)
   a[i][j]=rand()%200-100;
fw=fopen("pole.txt","w");
```



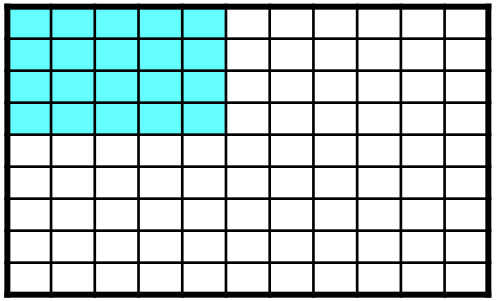
pole.txt



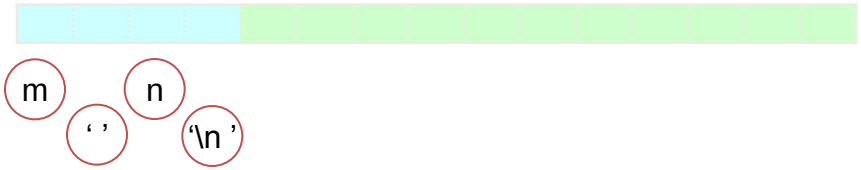
Dvojrozmerné pole , textový súbor – zápis a načítanie - príklad

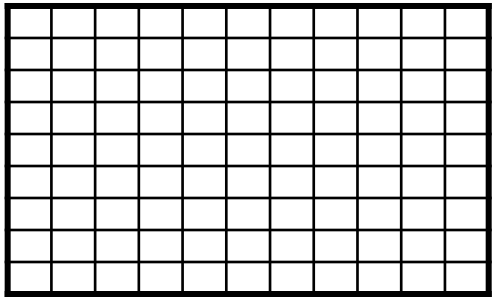
```
#define M 100
#define N 100
int a[M][N],b[M][N];
```

```
FILE *fw;
int m,n,i,j;
scanf("%d %d",&m,&n);
for(i=0;i<m;i++)
 for(j=0;j<n;j++)
   a[i][j]=rand()%200-100;
fw=fopen("pole.txt","w");
fprintf(fw,"%d %d\n",m,n);
```



pole.txt

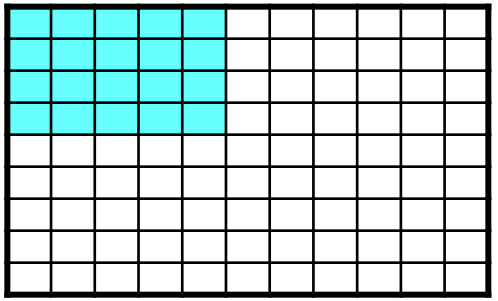




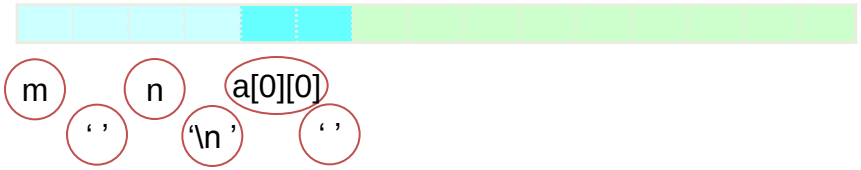
Dvojrozmerné pole , textový súbor – zápis a načítanie - príklad

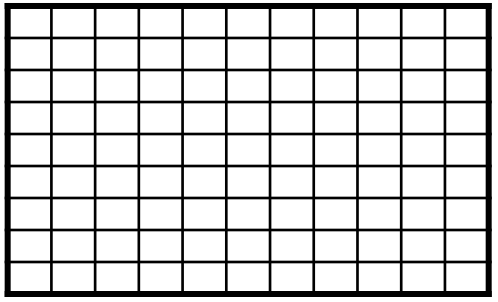
```
#define M 100
#define N 100
int a[M][N],b[M][N];
```

```
FILE *fw;
int m,n,i,j;
scanf("%d %d",&m,&n);
for(i=0;i<m;i++)
 for(j=0;j<n;j++)
   a[i][j]=rand()%200-100;
fw=fopen("pole.txt","w");
fprintf(fw,"%d %d\n",m,n);
for(i=0;i<m;i++)
 for(j=0;j<n;j++)
   fprintf(fw,"%d ",a[i][j]);
```



pole.txt

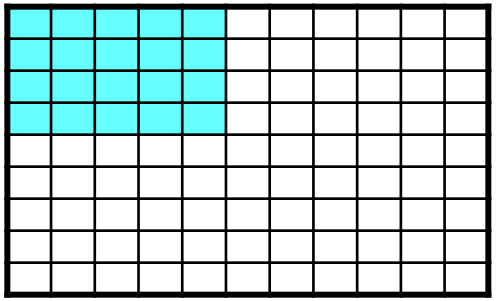




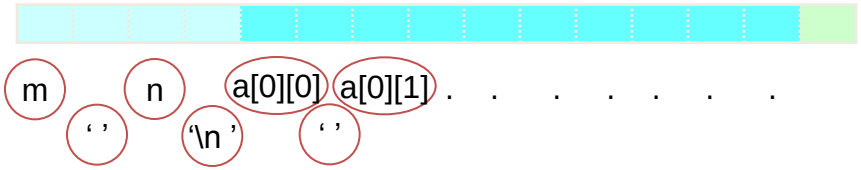
Dvojrozmerné pole , textový súbor – zápis a načítanie - príklad

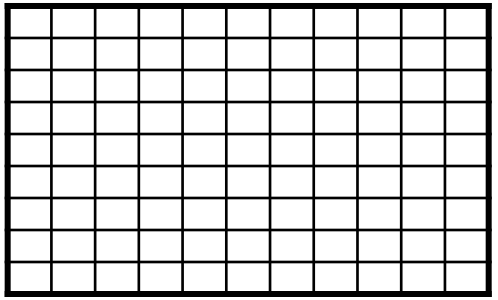
```
#define M 100
#define N 100
int a[M][N],b[M][N];
```

```
FILE *fw;
int m,n,i,j;
scanf("%d %d",&m,&n);
for(i=0;i<m;i++)
 for(j=0;j<n;j++)
   a[i][j]=rand()%200-100;
fw=fopen("pole.txt","w");
fprintf(fw,"%d %d\n",m,n);
for(i=0;i<m;i++)
 for(j=0;j<n;j++)
   fprintf(fw,"%d ",a[i][j]);
```



pole.txt

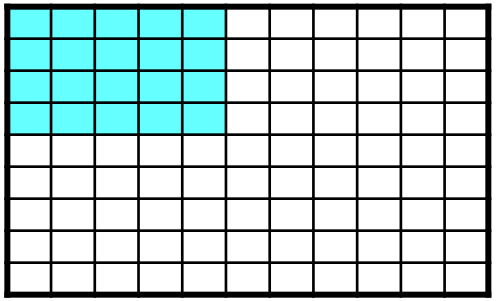




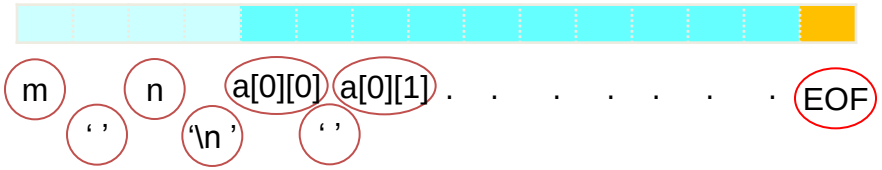
Dvojrozmerné pole , textový súbor – zápis a načítanie - príklad

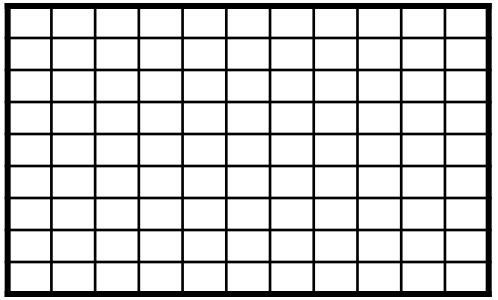
```
#define M 100
#define N 100
int a[M][N],b[M][N];
```

```
FILE *fw;
int m,n,i,j;
scanf("%d %d",&m,&n);
for(i=0;i<m;i++)
 for(j=0;j<n;j++)
   a[i][j]=rand()%200-100;
fw=fopen("pole.txt","w");
fprintf(fw,"%d %d\n",m,n);
for(i=0;i<m;i++)
 for(j=0;j<n;j++)
   fprintf(fw,"%d ",a[i][j]);
fclose(fw);
```



pole.txt



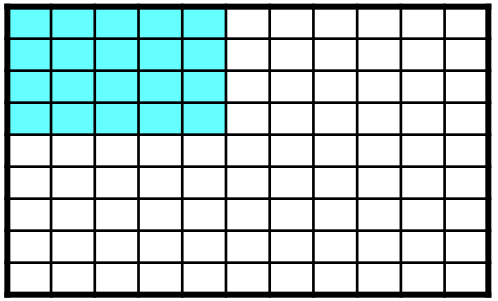


Dvojrozmerné pole , textový súbor – zápis a načítanie - príklad

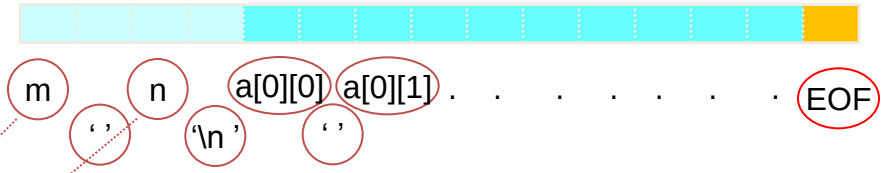
```
#define M 100
#define N 100
int a[M][N],b[M][N];
```

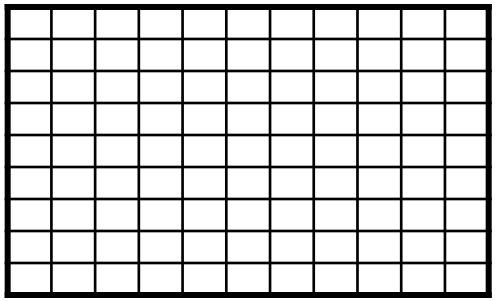
```
FILE *fw;
int m,n,i,j;
scanf("%d %d",&m,&n);
for(i=0;i<m;i++)
 for(j=0;j<n;j++)
   a[i][j]=rand()%200-100;
fw=fopen("pole.txt","w");
fprintf(fw,"%d %d\n",m,n);
for(i=0;i<m;i++)
 for(j=0;j<n;j++)
   fprintf(fw,"%d ",a[i][j]);
fclose(fw);
```

```
FILE *fr;
int m,n,i,j;
fr=fopen("pole.txt","r");
fscanf(fr,"%d %d",&m,&n);
```



pole.txt



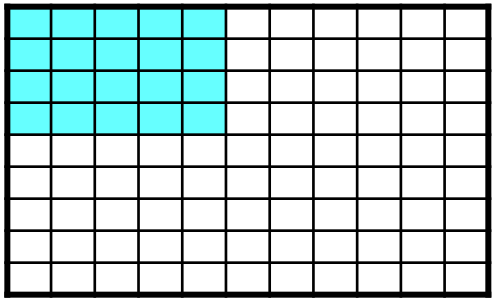


Dvojrozmerné pole , textový súbor – zápis a načítanie - príklad

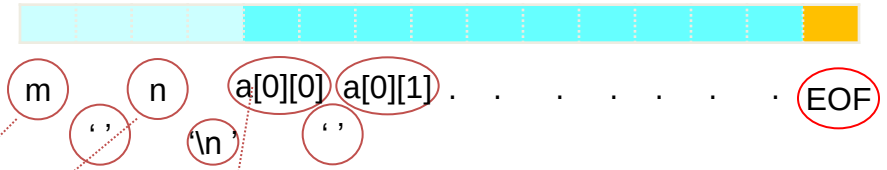
```
#define M 100
#define N 100
int a[M][N],b[M][N];
```

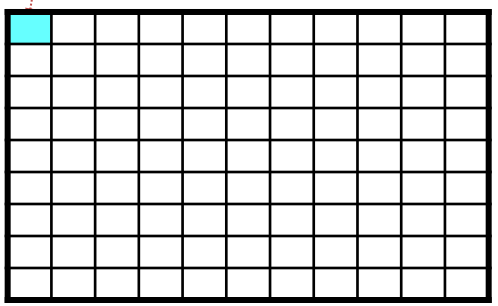
```
FILE *fw;
int m,n,i,j;
scanf("%d %d",&m,&n);
for(i=0;i<m;i++)
 for(j=0;j<n;j++)
   a[i][j]=rand()%200-100;
fw=fopen("pole.txt","w");
fprintf(fw,"%d %d\n",m,n);
for(i=0;i<m;i++)
 for(j=0;j<n;j++)
   fprintf(fw,"%d ",a[i][j]);
fclose(fw);
```

```
FILE *fr;
int m,n,i,j;
fr=fopen("pole.txt","r");
fscanf(fr,"%d %d",&m,&n);
for(i=0;i<m;i++)
 for(j=0;j<n;j++)
   fscanf(fr,"%d ",&b[i][j]);
```



pole.txt



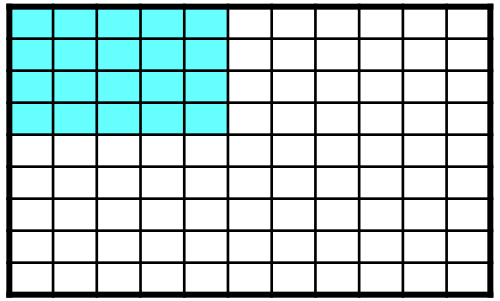


Dvojrozmerné pole , textový súbor – zápis a načítanie - príklad

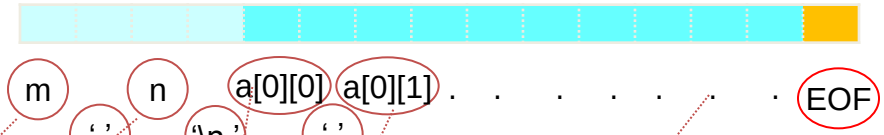
```
#define M 100
#define N 100
int a[M][N],b[M][N];
```

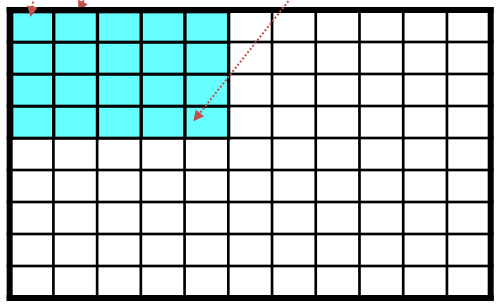
```
FILE *fw;
int m,n,i,j;
scanf("%d %d",&m,&n);
for(i=0;i<m;i++)
 for(j=0;j<n;j++)
   a[i][j]=rand()%200-100;
fw=fopen("pole.txt","w");
fprintf(fw,"%d %d\n",m,n);
for(i=0;i<m;i++)
 for(j=0;j<n;j++)
   fprintf(fw,"%d ",a[i][j]);
fclose(fw);
```

```
FILE *fr;
int m,n,i,j;
fr=fopen("pole.txt","r");
fscanf(fr,"%d %d",&m,&n);
for(i=0;i<m;i++)
 for(j=0;j<n;j++)
   fscanf(fr,"%d ",&b[i][j]);
```



pole.txt



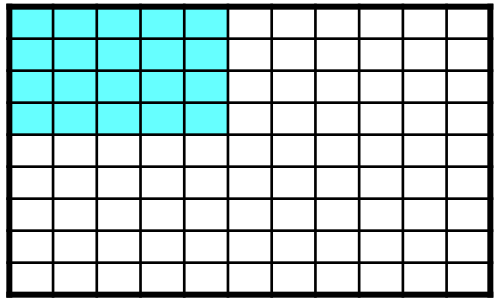


Dvojrozmerné pole , textový súbor – zápis a načítanie P7.3

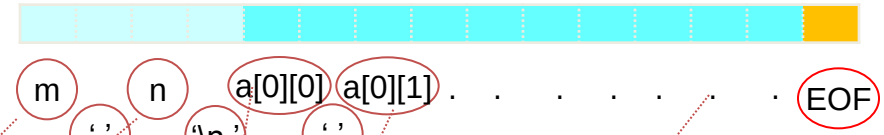
```
#define M 100
#define N 100
int a[M][N],b[M][N];
```

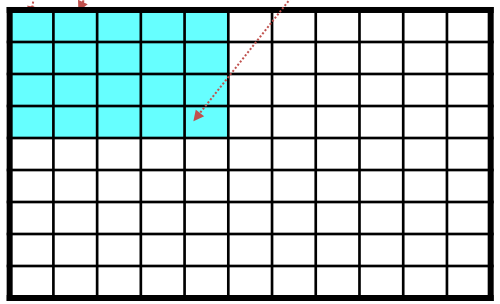
```
FILE *fw;
int m,n,i,j;
scanf("%d %d",&m,&n);
for(i=0;i<m;i++)
 for(j=0;j<n;j++)
   a[i][j]=rand()%200-100;
fw=fopen("pole.txt","w");
fprintf(fw,"%d %d\n",m,n);
for(i=0;i<m;i++)
 for(j=0;j<n;j++)
   fprintf(fw,"%d ",a[i][j]);
fclose(fw);
```

```
FILE *fr;
int m,n,i,j;
fr=fopen("pole.txt","r");
fscanf(fr,"%d %d",&m,&n);
for(i=0;i<m;i++)
 for(j=0;j<n;j++)
   fscanf(fr,"%d ",&b[i][j]);
fclose(fr);
for(i=0;i<m;i++)
 for(j=0;j<n;j++)
   printf("%d ",b[i][j]);
```



pole.txt





Pole smerníkov

```
int *xb[2];
```

- pole **xb**:
  - jednorozmerné pole dvoch smerníkov na **int**
  - smerníky sa využijú na riadky poľa, pre ktoré musíme alokovať pamäť pre 3 stĺpce

```
xb[0] = (int *) malloc(3* sizeof(int));
xb[1] = (int *) malloc(3* sizeof(int));
```

potom sa dá použiť:

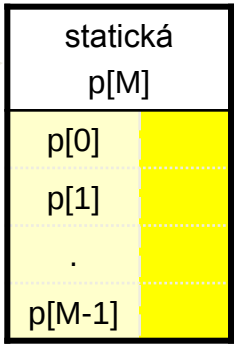
```
xb[0][2] = 5;
```

Dynamická alokácia dvojrozmerného poľa – pole smerníkov

```
#define M 4
#define N 3

typ *p[M];
```

Pole smerníkov

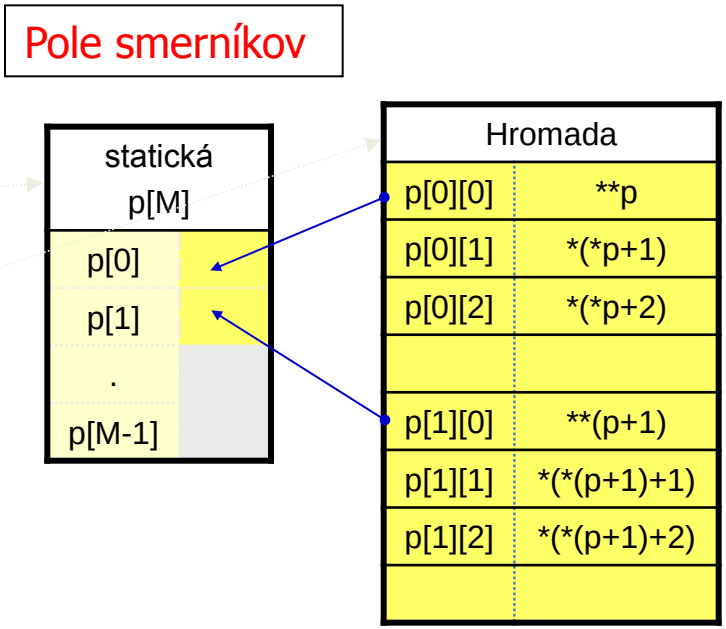


Dynamická alokácia dvojrozmerného poľa – pole smerníkov

```
#define M 4
#define N 3

typ *p[M];

m = 2;
n = 3;
for(i=0;i<m;i++)
  p[i]=(typ*)malloc(n*sizeof(typ));
```

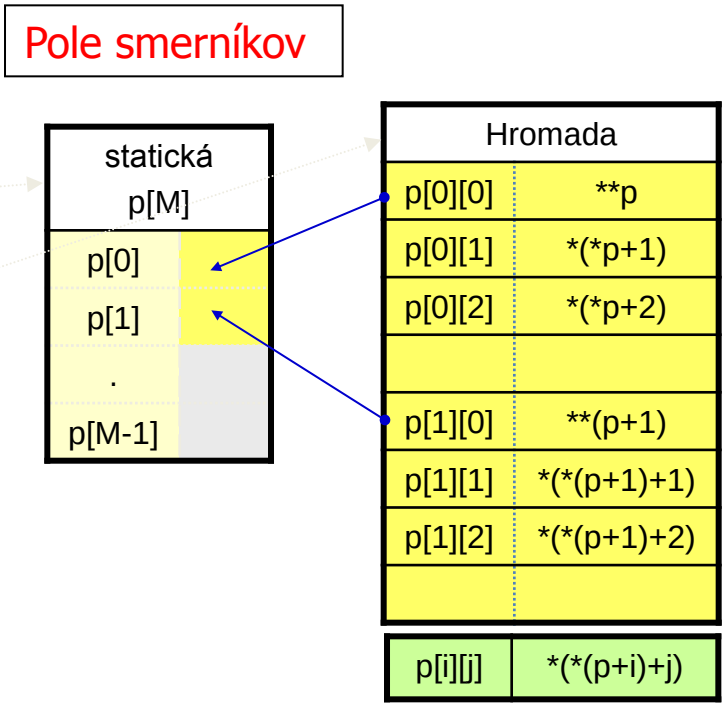


Dynamická alokácia dvojrozmerného poľa – pole smerníkov

```
#define M 4
#define N 3

typ *p[M];

m = 2;
n = 3;
for(i=0;i<m;i++)
  p[i]=(typ*)malloc(n*sizeof(typ));
```

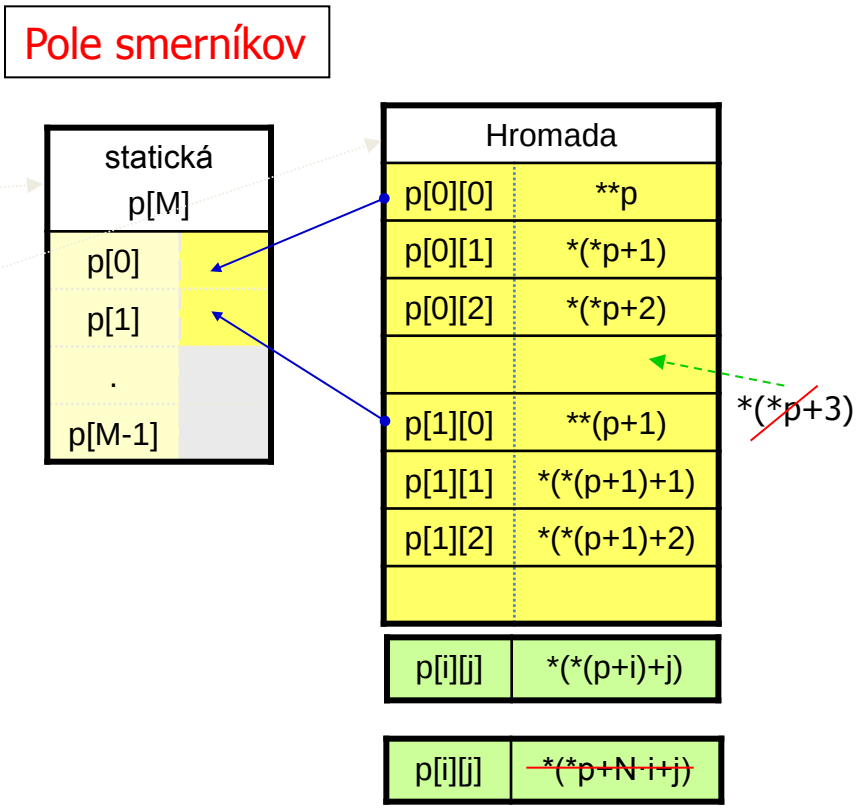


Dynamická alokácia dvojrozmerného poľa – pole smerníkov

```
#define M 4
#define N 3

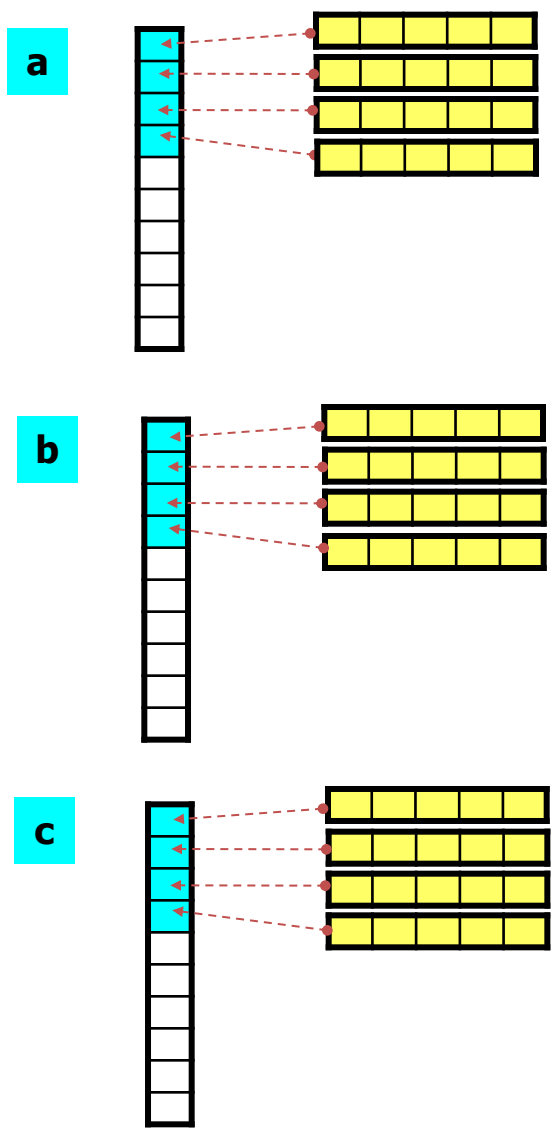
typ *p[M];

m = 2;
n = 3;
for(i=0;i<m;i++)
  p[i]=(typ*)malloc(n*sizeof(typ));
```



Súčet prvkov dvoch 2D polí pole smerníkov – príklad P7.4

```
#define M 100
#define N 100
int *a[M],*b[M],*c[M];
void citaj(int* x[],int m,int n)
{
  int i,j;
  for(i=0;i<m;i++) {
    x[i]=(int *)malloc(n*sizeof(int));
    for(j=0;j<n;j++) scanf("%d ",&x[i][j]);
  }
}
void sucet(int* x[],int *y[],int *z[],int m,int n)
{
  int i,j;
  for(i=0;i<m;i++) {
    z[i]=(int *)malloc(n*sizeof(int));
    for(j=0;j<m;j++) z[i][j]=x[i][j]+y[i][j];
  }
}
void tlac(int* x[],int m,int n)
{
  . . .
}
void main(void)
{  int m,n;
  scanf("%d %d",&m,&n);
  citaj(a,m,n);
  citaj(b,m,n);
  sucet(a,b,c,m,n);
  tlac(c,m,n);
}
```



Smerník na pole

```
int (*xc)[3];
```

- pole **xc**:
  - **xc** je smerník na pole troch **int**-ov
  - ak alokujeme dostatok pamäte - ako dvojrozmerné pole

```
xc = (int *) malloc(2 * 3 * sizeof(int));
```

- **xc** ukazuje na pole 6 prvkov združených po t
- obdoba statického poľa

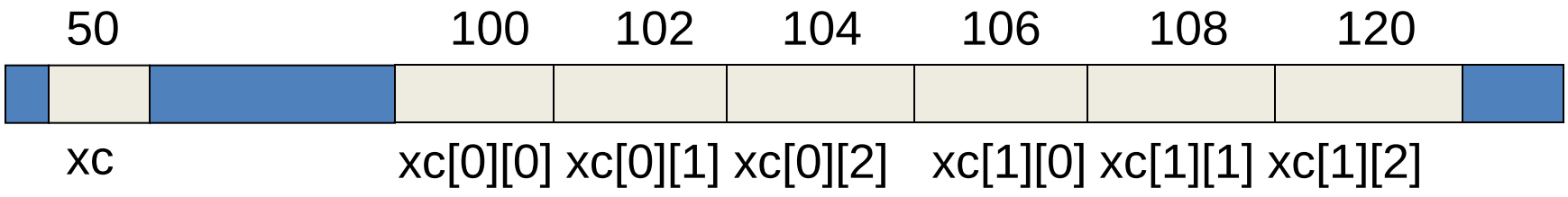
dá sa použiť:

```
xb[0][2] = 5;
```

Smerník na pole

```
int (*xc)[3];
```

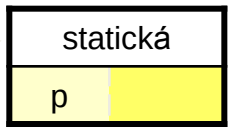
- pole **xc**:
  - jednotlivé riadky nasledujú v pamäti bezprostredne za sebou



Smerník na pole

```
#define M 4
#define N 3

typ (*p)[N];
```

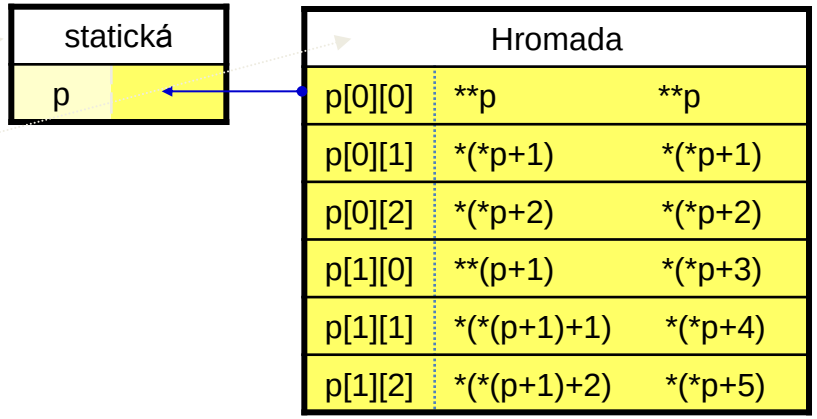


Smerník na pole

```
#define M 4
#define N 3

typ (*p)[N];

m=2;
p=(typ (*)[N])malloc(m*N*sizeof(typ));
```

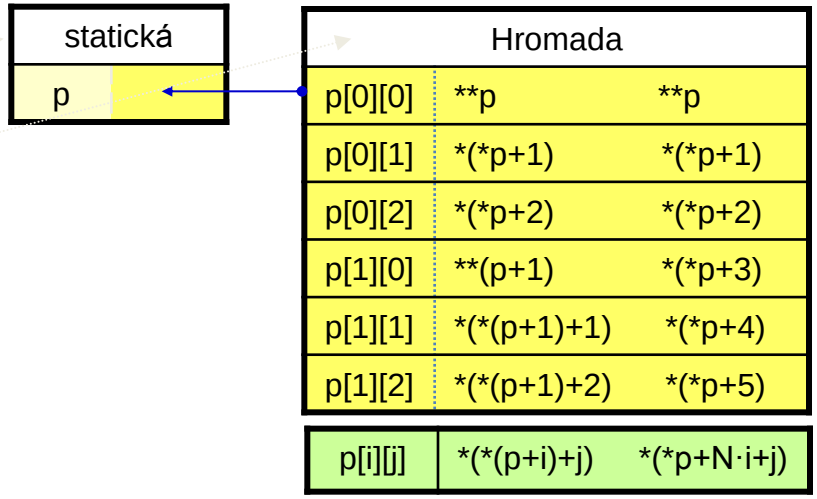


Smerník na pole

```
#define M 4
#define N 3

typ (*p)[N];

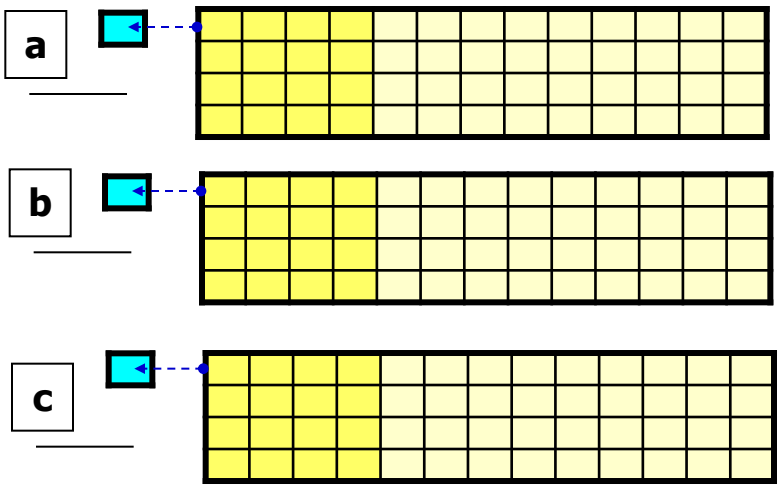
m=2;
p=(typ (*)[N])malloc(m*N*sizeof(typ));
```



Súčet prvkov dvoch 2D polí - smerník na pole – príklad P7.5

```
#define M 100
#define N 100
int (*a)[N],(*b)[N],(*c)[N];
int (*citaj(int m,int n))[]
{
 int i,j;
 int(*x)[N]=(int (*)[N])malloc(m*N*sizeof(int));
 for(i=0;i<m;i++)
  for(j =0;j<n;j++) scanf("%d",&x[i][j]);
 return x;
}
int (*sucet(int(*x)[N],int(*y)[N],int m,int n))[]
{
 int i,j;
 int(*z)[N]=(int(*)[N])malloc(m*N*sizeof(int));
 for(i=0;i<m;i++)
  for(j=0;j<n;j++)z[i][j]=x[i][j]+y[i][j];
 return z;
}
void tlac(int (*x)[N],int m,int j)
{
  . . .
}

void main(void)
{  int m,n;
  scanf("%d %d",&m,&n);
  a=citaj(m,n);
  b=citaj(m,n)
  c=sucet(a,b,m,n);
  tlac(c,m,n);
}
```



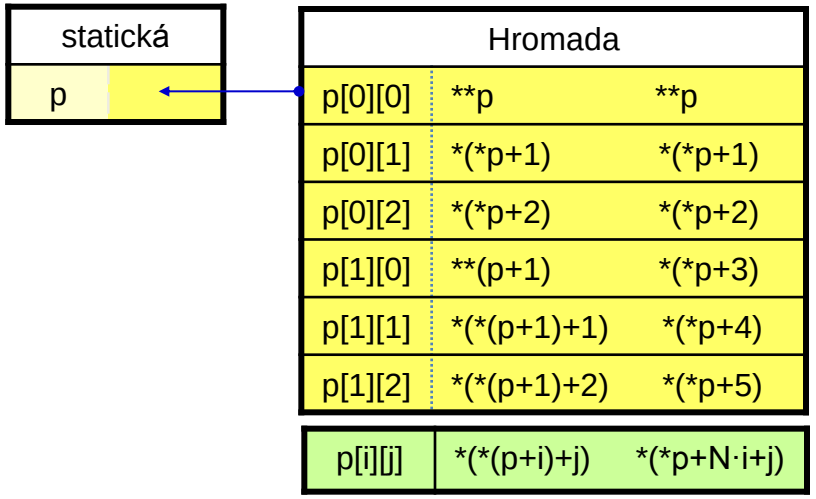
Porovnanie

Smerník na pole

```
#define M 4
#define N 3

typ (*p)[N];

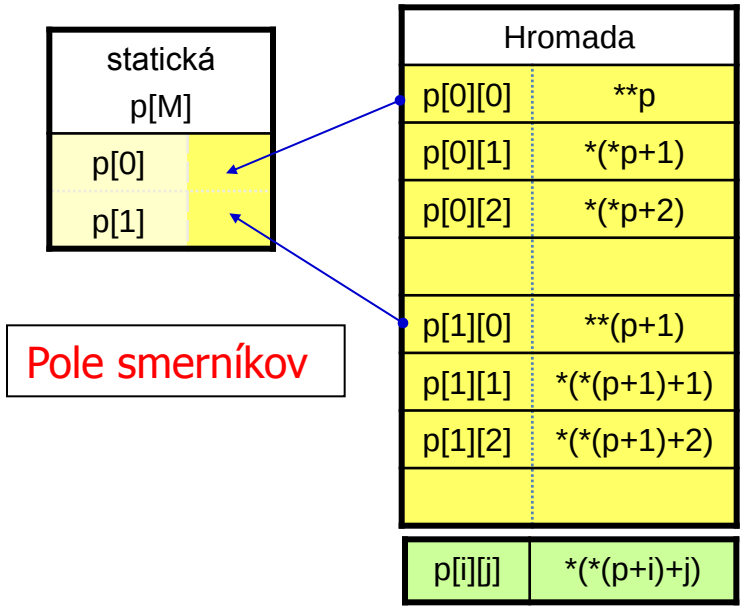
m=2;
p=(typ (*)[N])malloc(m*N*sizeof(typ));
```




```
#define M 4
#define N 3

typ *p[M];

m = 2;
n = 3;
for(i=0;i<m;i++)
  p[i]=(typ*)malloc(n*sizeof(typ));
```



Pole smerníkov

Smerník na smerník

```
int **xd;
```

- pole **xd**:
  - smerník na smerník, preto

(1) alokujeme ukazovatele na riadky

```
xd = (int **) malloc(2 sizeof(int *));
```

(2) alokujeme jednotlivé riadky

```
xd[0] = (int *) malloc(3* sizeof(int));
xd[1] = (int *) malloc(3* sizeof(int));
```

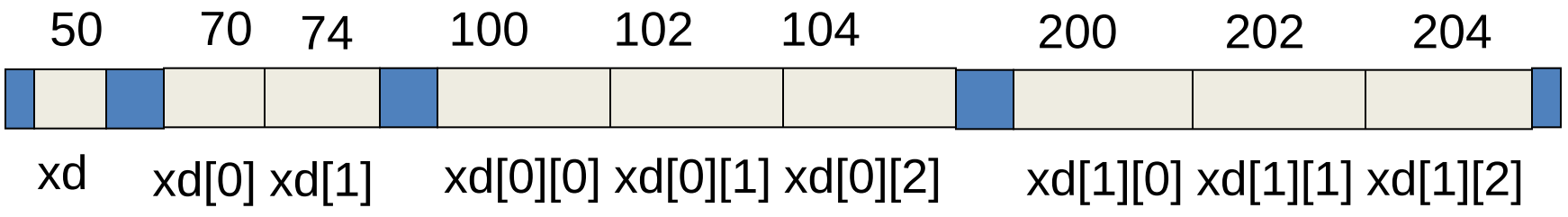
potom sa dá použiť:

```
xd[0][2] = 5;
```

Smerník na smerník

```
int **xd;
```

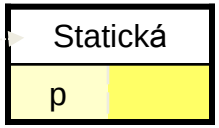
- pole **xd**:
  - **xd** je ukazovateľ na ukazovateľ na typ **int**
  - ***xd** je ukazovateľ na typ **int**
  - ****xd** je prvok typu **int**



Smerník na smerník

```
#define M 4
#define N 3

typ **p;
```

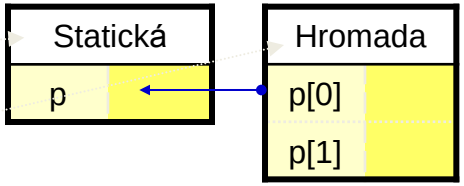


```
#define M 4
#define N 3

typ **p;

n = 3;
m = 2;
p=(typ**)malloc(m*sizeof(typ*));
```

Smerník na smerník

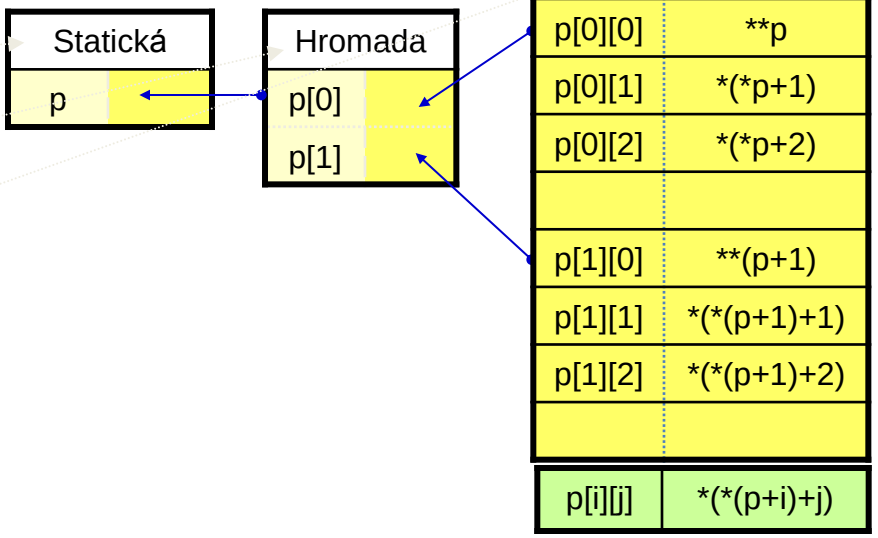


```
#define M 4
#define N 3

typ **p;

n = 3;
m = 2;
p=(typ**)malloc(m*sizeof(typ*));
for(i=0;i< m;i++)
  p[i]=(typ*)malloc(n*sizeof(typ));
```

Smerník na smerník



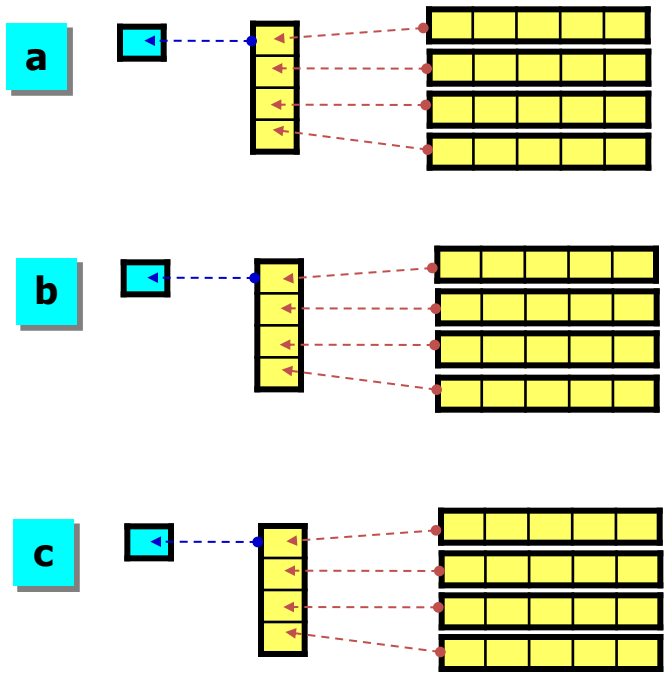
Súčet prvkov dvoch 2D polí - smerník na smerník – príklad P7.6

```
#define M 100
#define N 100
int **a,**b,**c;

int **citaj(int m,int n)
{
 int i,j,**x=(int **)malloc(m*sizeof(int*));
 for(i=0;i<m;i++) {
  x[i]=(int *)malloc(n*sizeof(int));
  for(j=0;j<n;j++) scanf("%d",&x[i][j]);
 }
 return x;
}

int **sucet(int **x,int **y,int m,int n)
{
 int i,j,**z=(int **)malloc(m*sizeof(int));
 for(i=0;i<m;i++) {
  z[i]=(int *)malloc(n*sizeof(int));
  for(j=0;j<n;j++)z[i][j]=x[i][j]+y[i][j];
 }
 return z;
}
void tlac(int **x,int m,int j)
{
  .
}

void main(void)
{  int m,n;
   scanf("%d %d",&m,&n);
   a=citaj(m,n);
   b=citaj(m,n);
   c=sucet(a,b,m,n);
   tlac(c,m,n);
}
```

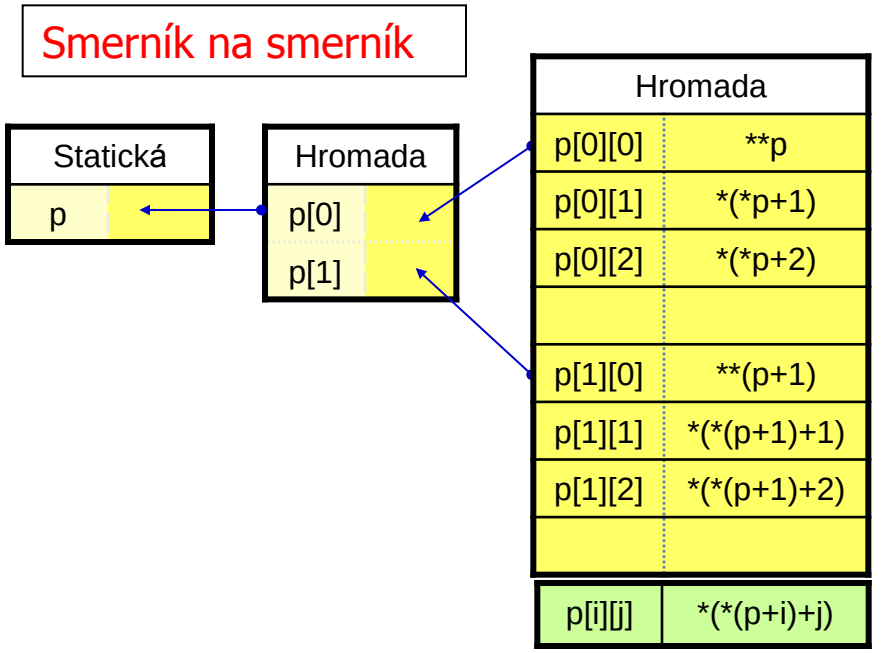


Porovnanie

```
#define M 4
#define N 3

typ **p;

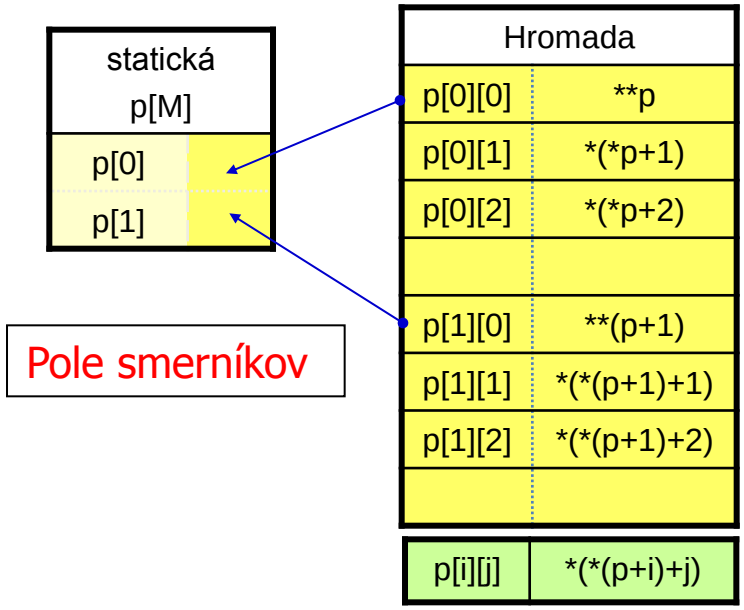
n = 3;
m = 2;
p=(typ**)malloc(m*sizeof(typ*));
for(i=0;i< m;i++)
  p[i]=(typ*)malloc(n*sizeof(typ));
```




```
#define M 4
#define N 3

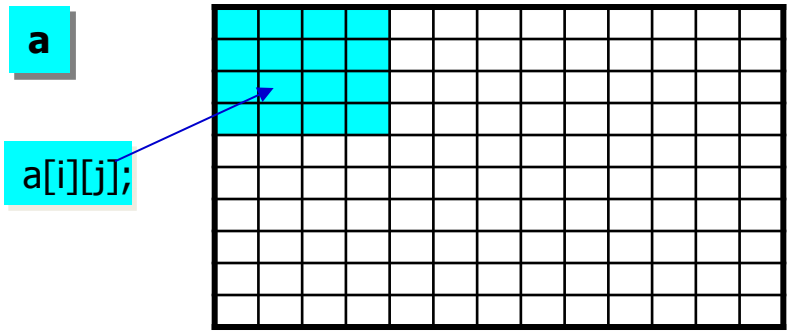
typ *p[M];

m = 2;
n = 3;
for(i=0;i<m;i++)
  p[i]=(typ*)malloc(n*sizeof(typ));
```

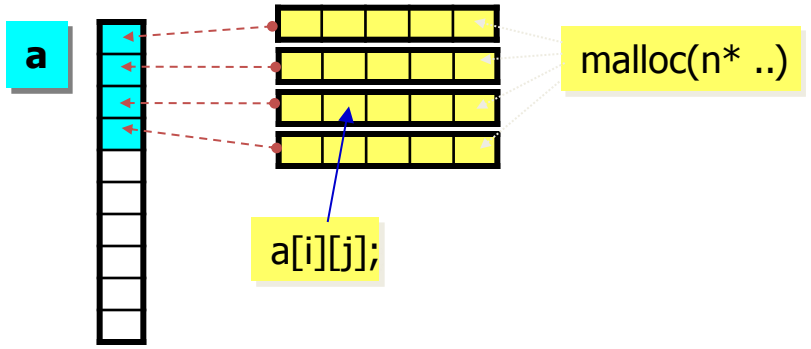


Shrnutie - možnosti definovania 2D polí

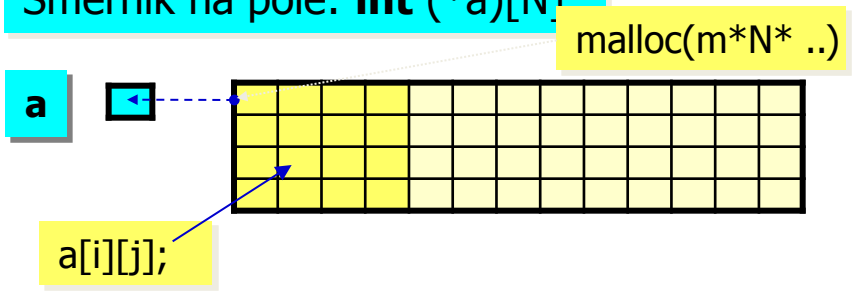
Statické pole: **int** a[M][N];



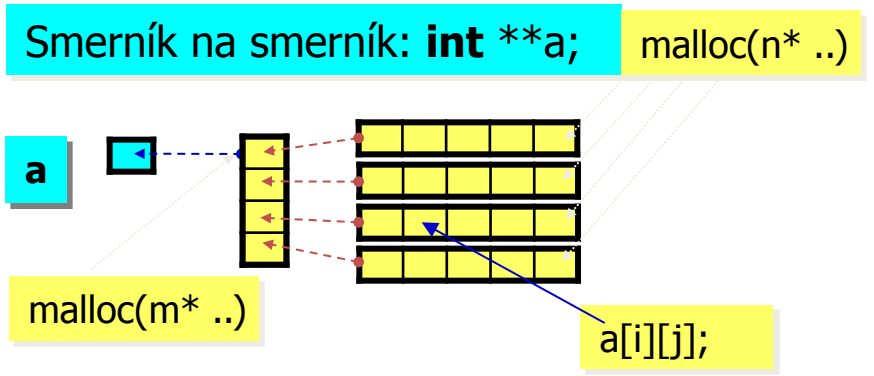
Pole smerníkov: **int** *a[M];



Smerník na pole: **int** (*a)[N];

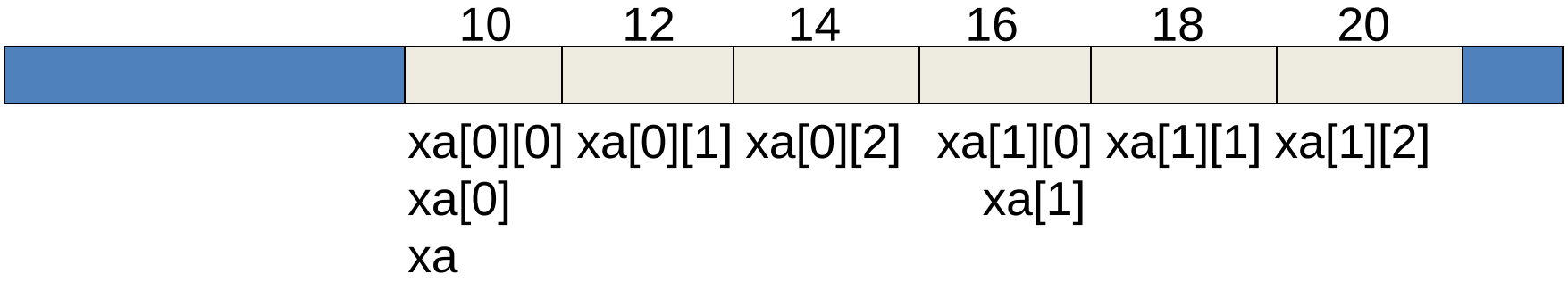


Smerník na smerník: **int** **a;



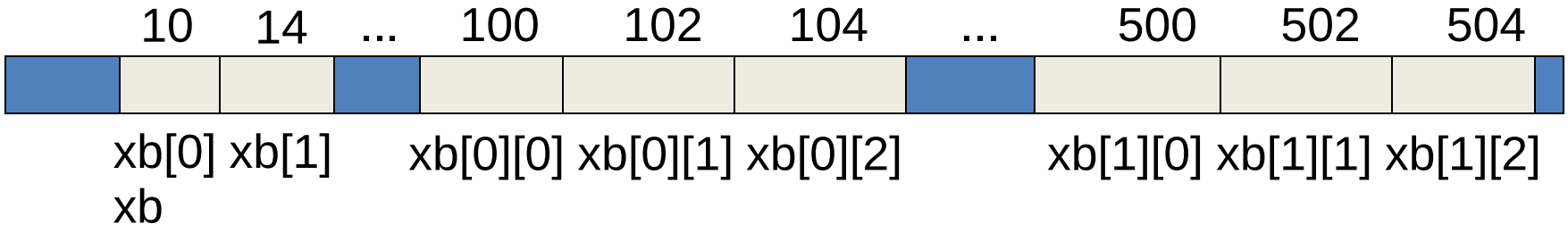
Výhody a nevýhody spôsobov vytvárania polí: pamäť. nároky

- **`xa`** (**`int xa[2][3]`**): pamäťovo najvýhodnejšia



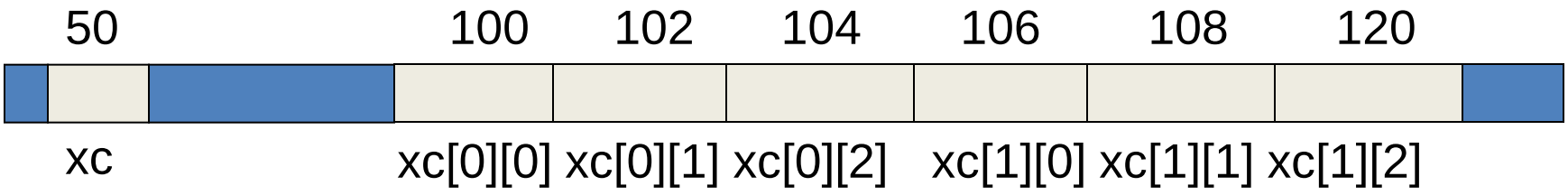
Výhody a nevýhody spôsobov vytvárania polí: pamäť. nároky

- **`xa`** (**`int xa[2][3]`**): pamäťovo najvýhodnejšia
- **`xb`** (**`int *xb[2]`**): naviac pamäť pre 2 ukazovatele (počet riadkov **`xb[0]`**, **`xb[1]`**)



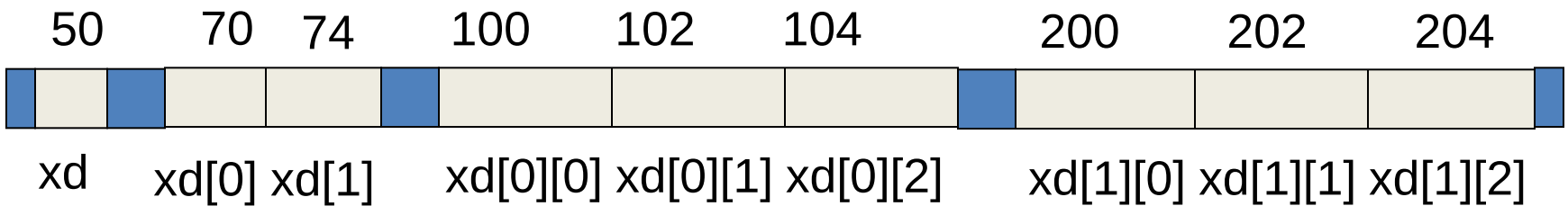
Výhody a nevýhody spôsobov vytvárania polí: pamäť. nároky

- **`xa`** (**`int xa[2][3]`**): pamäťovo najvýhodnejšia
- **`xb`** (**`int *xb[2]`**): naviac pamäť pre 2 ukazovatele (počet riadkov **`xb[0]`**, **`xb[1]`**)
- **`xc`** (**`int (*xc)[3]`**): naviac pamäť pre 1 ukazovateľ na typ **`int`**: **`xc`**



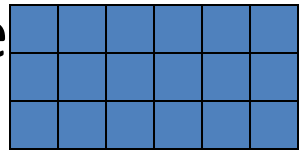
Výhody a nevýhody spôsobov vytvárania polí: pamäť. nároky

- **`xa`** (**`int xa[2][3]`**): pamäťovo najvýhodnejšie
- **`xb`** (**`int *xb[2]`**): naviac pamäť pre 2 ukazovatele (počet riadkov **`xb[0]`**, **`xb[1]`**)
- **`xc`** (**`int (*xc)[3]`**): naviac pamäť pre 1 ukazovateľ na typ **`int`**: **`xc`**
- **`xd`** (**`int **xd`**): naviac 1 ukazovateľ pre **`xd`** a 2 ukazovatele na riadky (**`xd[0], xd[1]`**)

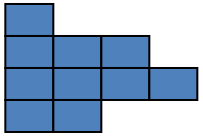


Výhody a nevýhody spôsobov vytvárania polí: charakter

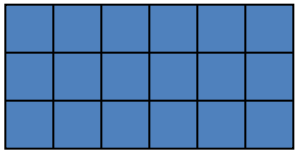
- **`xa`**(**`int xa[3][6]`**): pravoúhle pole



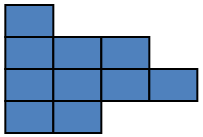
- **`xb`**(**`int *xb[4]`**): "zubaté" pole



- **`xc`** (**`int (*xc)[6]`**): pravoúhle pole



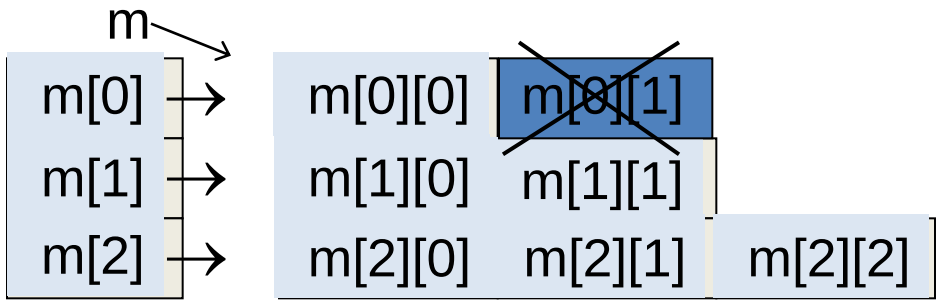
- **`xd`**(**`int **xd`**): "zubaté" pole



príklad "zubaté" pole

- dvojrozmerné pole s rôznou dĺžkou riadkov - časť matice pod diagonálou (vrátane) - **`int *m[3]`**

```
int *m[3], i;
for(i = 0; i < 3; i++)
    m[i] = (int *) malloc((i+1) * sizeof(int));
```



príklad "zubaté" pole

```
int **r = malloc(3*sizeof(int*));
r[0] = malloc(2*sizeof(int));
r[1] = malloc(3*sizeof(int));
r[2] = malloc(1*sizeof(int));
```

r[0][0] r[0][1]
r[1][0] r[1][1] r[1][2]
r[2][0]

Dvojrozmerné pole ako parameter funkcie

- ako jednorozmerné pole
- odlišnosť:
  - prvá dimenzia - prázdna []
  - druhá dimenzia musí byť uvedená, napr. [10]
- preto
  - je potrebné preniesť do funkcie aj počet riadkov
  - skutočný parameter: len pravouhlé polia (**`xa`**, **`xc`**)

pre: **`double x[5][6];`**

`double x[][6]` alebo **`double (*x)[6]`**

~~**`double *x[6]`**~~

Dvojrozmerné pole ako parameter funkcie P7.7

funkcia vráti maximum z prvkov dvojrozmerného poľa

```
double maximum(double pole [][4], int riadky)
{
    double pom = pole[0][0];
    int i, j;

    for (i = 0; i < riadky; i++) {
        for (j = 0; j < 4; j++)
            if (pole[i][j] > pom)
                pom = pole[i][j];
    }
    return (pom);
 }
```